Lava Phase-1 Release Rotation Runbook

Operator runbook for rotating the per-build UUID + pepper that Phase-1's client-auth foundation embeds in each Android release. The auth flow is covered by:

- `docs/superpowers/specs/2026-05-06-phase1-api-auth-design.md` — design
- `docs/superpowers/plans/2026-05-06-phase1-api-auth.md` — plan
- `core/CLAUDE.md` Auth UUID memory hygiene clause
- §6.R No-Hardcoding Mandate (root `CLAUDE.md`)

The TL;DR: every Android release ships with a unique UUID (the per-build client identifier) AND a unique obfuscation pepper (rotates the build-time AES key derivation). The lava-api-go service maintains an allowlist of accepted UUIDs (active) plus a separate retired list that returns `426 Upgrade Required` so old APKs prompt the user to update.

When to rotate

Rotate every release tag. The mechanism is engineered so reusing a previous (UUID, pepper) pair across releases REDUCES security: the single attacker decompile attack on version N also pwns version N+1.

A leaked UUID detected mid-cycle MUST be rotated AS SOON AS the leak is confirmed — see "Mid-cycle leak response" below.

Step-by-step rotation (every release)

1. Generate fresh UUID + pepper

```
NEW_UUID="$(uuidgen)"
NEW_PEPPER="$(openssl rand -base64 32)"
echo "UUID:   $NEW_UUID"
echo "Pepper: $NEW_PEPPER"
```

2. Decide the new client name

The naming convention is `android-${VERSION_NAME}-${VERSION_CODE}`, matching `app/build.gradle.kts` versionName + versionCode. Bump both in `app/build.gradle.kts` first if you haven't already (§6.P forbids re-distributing the same versionCode).

```
android-1.2.7-1027
```

3. Update `.env` (gitignored, per-operator)

Append the new entry to the active list, set the current name, and rotate the pepper:

```
LAVA_AUTH_CURRENT_CLIENT_NAME=android-1.2.7-1027
LAVA_AUTH_ACTIVE_CLIENTS=android-1.2.7-1027:$
        {NEW_UUID},android-1.2.6-1026:<previous-uuid>
LAVA_AUTH_RETIRED_CLIENTS=
LAVA_AUTH_OBFUSCATION_PEPPER=${NEW_PEPPER}
```

If a previously-active release should now be forced-upgrade (because ≥95% of users have already moved past it, OR because its UUID leaked), move that entry from `LAVA_AUTH_ACTIVE_CLIENTS` to `LAVA_AUTH_RETIRED_CLIENTS`.

The same `.env` is read by:

- the Android Gradle codegen (`generateLavaAuthClassDebug/Release`)
- the lava-api-go service at boot

So both sides see the same allowlist.

4. Build the APK

```
./gradlew :app:clean :app:assembleRelease
```

The `generateLavaAuthClassRelease` task generates `app/build/generated/lava-auth/main/lava/auth/LavaAuthGenerated.kt` containing the AES-GCM-encrypted UUID + nonce + pepper bytes + field name. The runtime `AuthInterceptor` decrypts on each request.

5. Build the API binary

```
( cd lava-api-go && make build && make image )
```

6. Stage the snapshot files

Per §6.P, every distribute action requires:

- A `CHANGELOG.md` entry referencing the new version
- A per-version snapshot file under `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/<version>-<code>.md`

Both are committed before the distribute script runs.

7. Distribute the API to thinker.local

```
./scripts/distribute-api-remote.sh
```

The script copies the new image to thinker.local, reloads the systemd unit, and verifies `/health` returns 200. The new `.env` is also synced so the API sees the rotated allowlist.

8. Distribute the APK via Firebase

```
./scripts/firebase-distribute.sh --release-only
```

This runs through:

- §6.P Gates 1+2+3 (versionCode monotonic, CHANGELOG entry, snapshot file)
- Phase-1 Gate 4 (pepper rotated — the script refuses if its SHA-256 is in `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/pepper-history.sha256`)
- Phase-1 Gate 5 (`LAVA_AUTH_CURRENT_CLIENT_NAME` matches `android-${APP_VERSION}-${APP_VERSION_CODE}` AND appears in `LAVA_AUTH_ACTIVE_CLIENTS`)

After upload, both `last-version` and `pepper-history.sha256` are updated so the next session is gated by them.

9. Tag the release

```
./scripts/tag.sh Lava-Android-1.2.7-1027
./scripts/tag.sh Lava-API-Go-2.1.0-2100
```

`scripts/tag.sh` requires §6.I matrix-gate evidence (`.lava-ci-evidence/Lava-Android-1.2.7-1027/real-device-verification.md`) to operate. See the §6.I clause in root `CLAUDE.md`.

10. Post-distribute audit

After ⩾95% of installs are on the new version (verifiable via Firebase Analytics or Crashlytics version distribution), retire the previous release:

```
# Move from ACTIVE to RETIRED
LAVA_AUTH_RETIRED_CLIENTS=android-1.2.6-1026:<previous-uuid>
LAVA_AUTH_ACTIVE_CLIENTS=android-1.2.7-1027:${NEW_UUID}
```

Restart lava-api-go so the new allowlist takes effect. Old APKs now receive `426 Upgrade Required` with min-version JSON; the client surfaces "please upgrade" dialogs.

Mid-cycle leak response

If an active UUID is leaked between releases (e.g. detected via API abuse, brute-force-rate spike, or a tester's APK reverse-engineered):

1. Generate a NEW UUID + pepper for an emergency release (steps 1-2 above).
2. Skip directly to step 8 (move the leaked entry from ACTIVE to RETIRED) BEFORE the new release ships. Old APKs now get `426`.
3. Build + distribute the new release ASAP per steps 4-9.

The window between (a) marking RETIRED and (b) shipping the new APK is when honest users on the leaked release see the upgrade dialog. Keep this window short.

Rotation contract guarantees

- **Re-signed-APK fail-closed.** The build-time HKDF salt is SHA-256(signing-cert-DER)[:16]. A re-signed APK has a different cert → different hash → different derived key → AES-GCM decrypt fails closed at request time (`AEADBadTagException` propagates as `IOException`).
- **Pepper rotation invalidates extraction.** A reverse-engineer who extracts the UUID + pepper from version N has N's encryption blob, but version N+1 ships a fresh pepper, so version N's plaintext doesn't decrypt N+1's blob. The window for an extracted secret is one release.
- **Active vs retired distinction.** A retired UUID returns 426 (no backoff increment — the user is honest, just outdated). An unknown UUID returns 401 + per-IP backoff (the per-key fixed-step ladder primitive in `submodules/ratelimiter/pkg/ladder`).

Constitutional bindings

- §6.R — every value in this runbook (UUIDs, peppers, allowlists) comes from `.env`, never tracked source.
- §6.H — `.env` is gitignored; never committed.
- §6.G — every Android release MUST pass the §6.I emulator-matrix gate before tagging.
- §6.J — the Compose UI Challenge Tests C15+C16 (Phase 13) are the load-bearing Sixth Law clause-4 acceptance gate for this auth feature.
- §6.P — `firebase-distribute.sh` enforces monotonic versionCode + CHANGELOG entry + snapshot presence.
- Phase-1 Gates 4+5 — pepper rotation + client-name consistency (added 2026-05-06).